

Cairo University

Faculty of Computers and Artificial Intelligence

CS251

**Introduction to
Software Engineering**

BudgetingSW

Software Design Specifications

Version 1.0

4 - 2026



CS251: Ducks in Hell

Project: BudgetingSW

Software Design Specification

Contents

Team.....	3
Document Purpose and Audience.....	3
I. Purpose.....	3
II. Audience.....	3
System Models.....	4
I. Architecture Diagram.....	4
A. Software Architecture: MVC + Design Patterns.....	4
B. System Components & Packages.....	4
II. Class Diagram(s).....	8
III. Class Descriptions.....	11
IV. Sequence diagrams.....	13
Class - Sequence Usage Table.....	21
V. State Diagram.....	23
VI. SOLID Principles.....	24
A. Single Responsibility Principle.....	24
B. Open/Close Principle.....	25
C. Dependency Inversion.....	26
VII. Design Patterns.....	27
A. Observer Pattern.....	27
B. Template Method Pattern.....	28
C. Strategy Pattern.....	29
Tools.....	30
Ownership Report.....	30



CS251: Ducks in Hell

Project: BudgetingSW

Software Design Specification

Team

ID	Name	Email	Mobile
20240354	Ali Wael	20240354@stud.fci-cu.com	01018182749
20240481	Mohamed Ayman	20240481@stud.fci-cu.com	01016111955
20240671	Youssef Ibrahim	20240671@stud.fci-cu.com	01119834356
20240376	Omar Abdelmonem	20240376@stud.fci-cu.com	01289033515

Document Purpose and Audience

I. Purpose

Providing diagrams that explain the system behaviour and the design of classes that the development team needs to implement

II. Audience

- Developer
- Project Manager



CS251: Ducks in Hell

Project: BudgetingSW

Software Design Specification

System Models

I. Architecture Diagram

A. Software Architecture: MVC + Design Patterns

The system follows a layered **MVC (Model-View-Controller)** architectural pattern built on JavaFX FXML, augmented with several design patterns, so that it achieves:

1. **Separation of concerns:** The View (FXML screens) is separate from business logic (Model managers) with Controllers acting as the bridge, ensuring minimal coupling between layers.
2. **Testability:** Each layer exposes its own interface, and design patterns like Strategy and Template Method allow swapping implementations without affecting other layers.
3. **Flexibility:** Adding new Views only requires a new Controller; changing internal logic inside any manager doesn't affect the View code. The Observer pattern further decouples managers from UI updates.

B. System Components & Packages

The system is divided into four primary components (packages):

- **View and Controller Layer:** Contains the JavaFX FXML screens (Login, SignUp, Dashboard, Transactions, Budget, Goals, Reports, Profile) and their corresponding Controllers (LoginController, DashboardController, TransactionsController, BudgetController, GoalsController, ReportsController, ProfileController), along with helpers like AvatarHelper, DialogHelper, and CurrencyUtil.
- **ApplicationState (Singleton):** The central coordinator that owns all managers, holds the session token, wires PropertyChange listeners, and broadcasts state change events to UI controllers.
- **Model Layer:** The business logic of the application, which includes:
 - **Authentication (Template Method + Strategy):** Abstract Recognition template with concrete Login and SignUp implementations; uses EncryptionUtil (AES) for password hashing and Session for token persistence
 - **TransactionManager (Observer Subject):** Validates and adds transactions, then fires PropertyChangeEvents to notify BudgetManager, SavingGoalsManager, and UI controllers



CS251: Ducks in Hell

Project: BudgetingSW

Software Design Specification

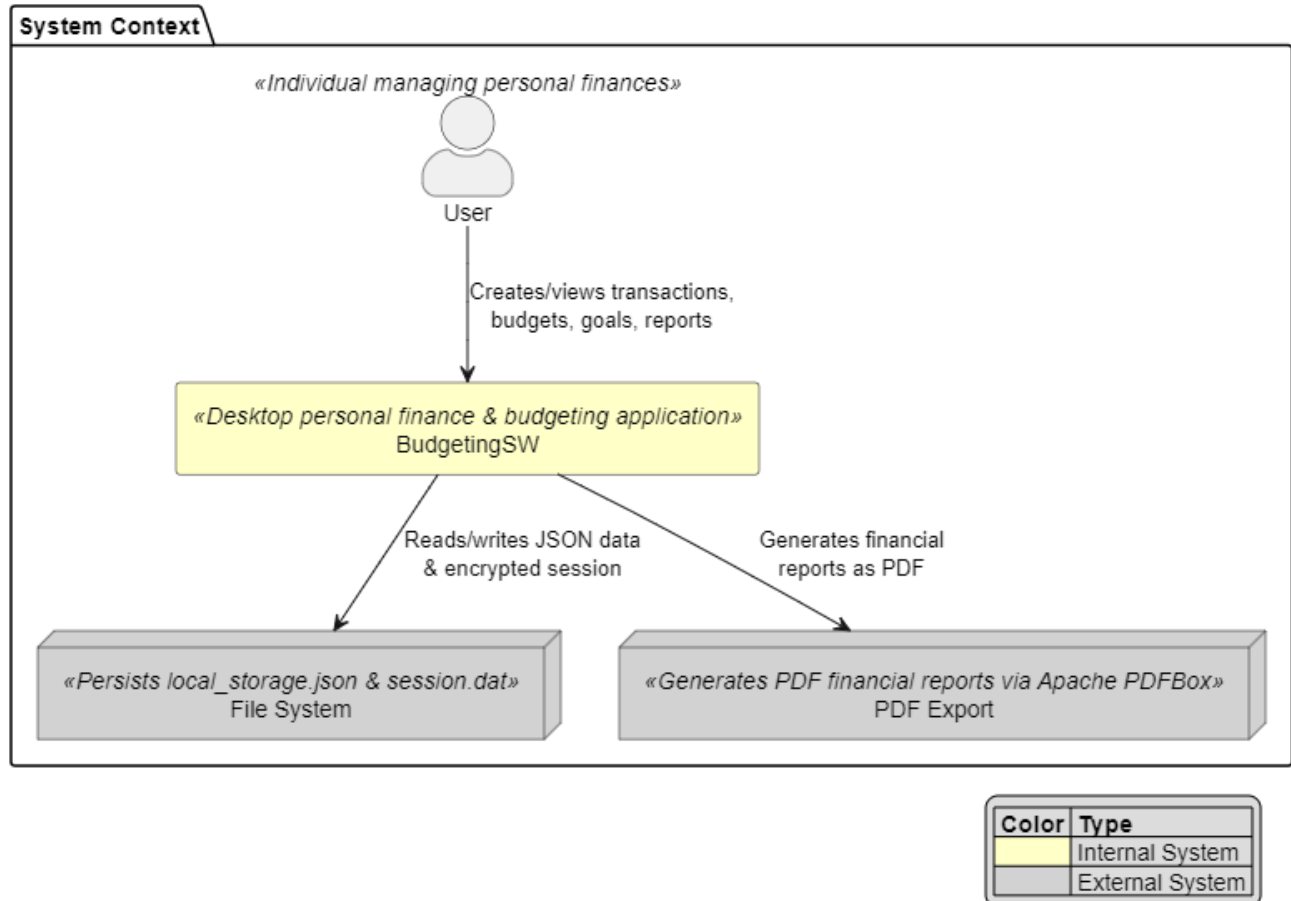
- BudgetManager (*Observer + Domain*): Creates and edits Budgets per account; listens to TransactionManager events and checks threshold alerts
- SavingGoalsManager (*Observer + Domain*): Creates SavingGoals and calculates monthly savings; listens to TransactionManager events and updates current amounts on new income
- AccountManager (*Account Operations*): Manages profile edits, password updates via EncryptionUtil, and account deletion
- PDFReport / ReportGenerator (*Strategy*): Queries TransactionManager with a TransactionConfig filter, aggregates results by category and percent, and exports to a PDF file
- **Data Accessing Layer:** Provides unified data persistence through:
 - LocalStorage (*Singleton, JSON*): Persists all data (Transactions, Accounts, Budgets, Categories, SavingGoals) in a local JSON file keyed by AppSettings. DataKey implements the StorageStrategy interface to allow swapping storage backends.
 - Domain Model (*POJOs*): Plain data objects shared across all layers, including Account, AccountConfig, AppSettings, Transaction, TransactionConfig, Budget, SavingGoal, Period, Range, and ReportConfig



CS251: Ducks in Hell

Project: BudgetingSW

Software Design Specification

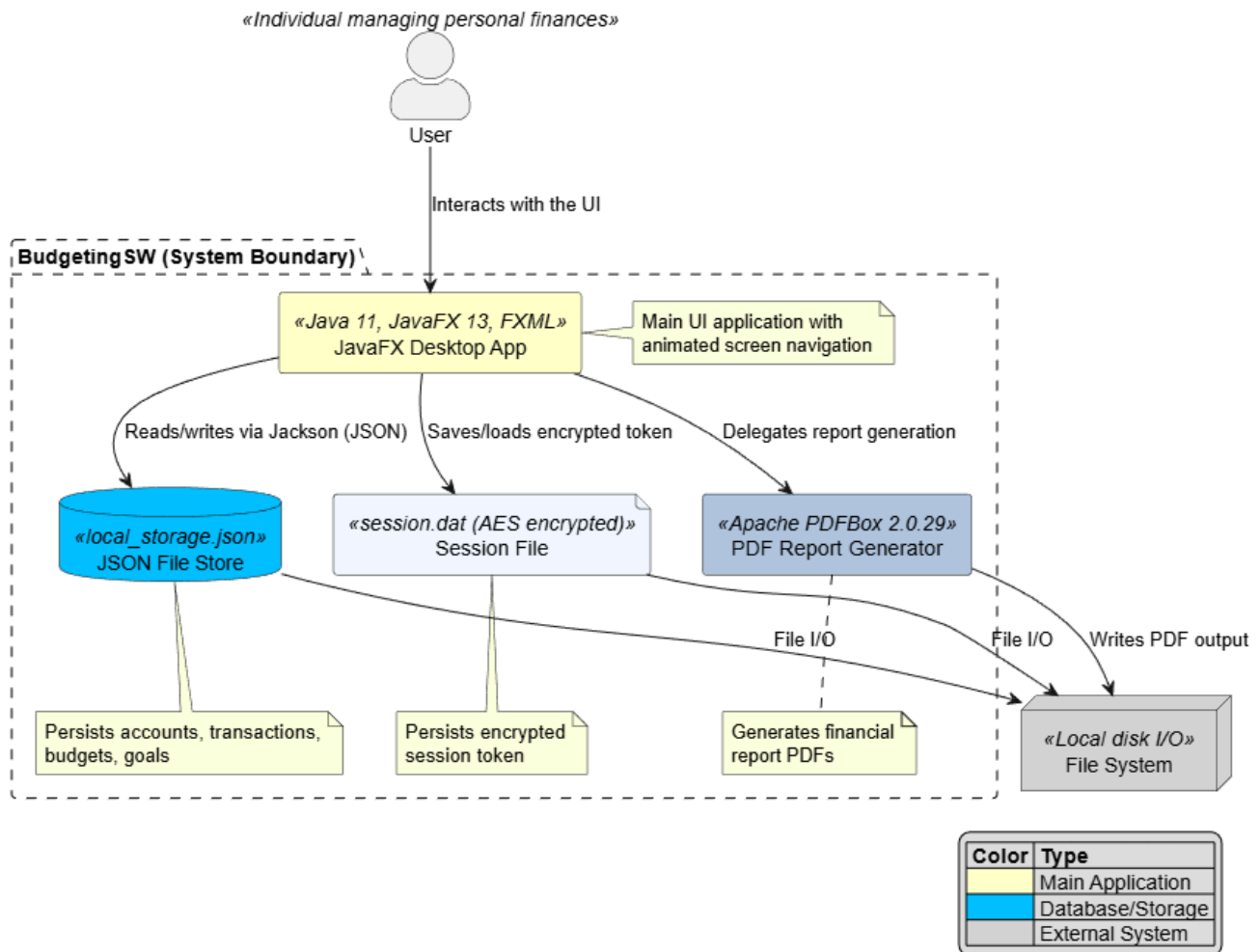




CS251: Ducks in Hell

Project: BudgetingSW

Software Design Specification

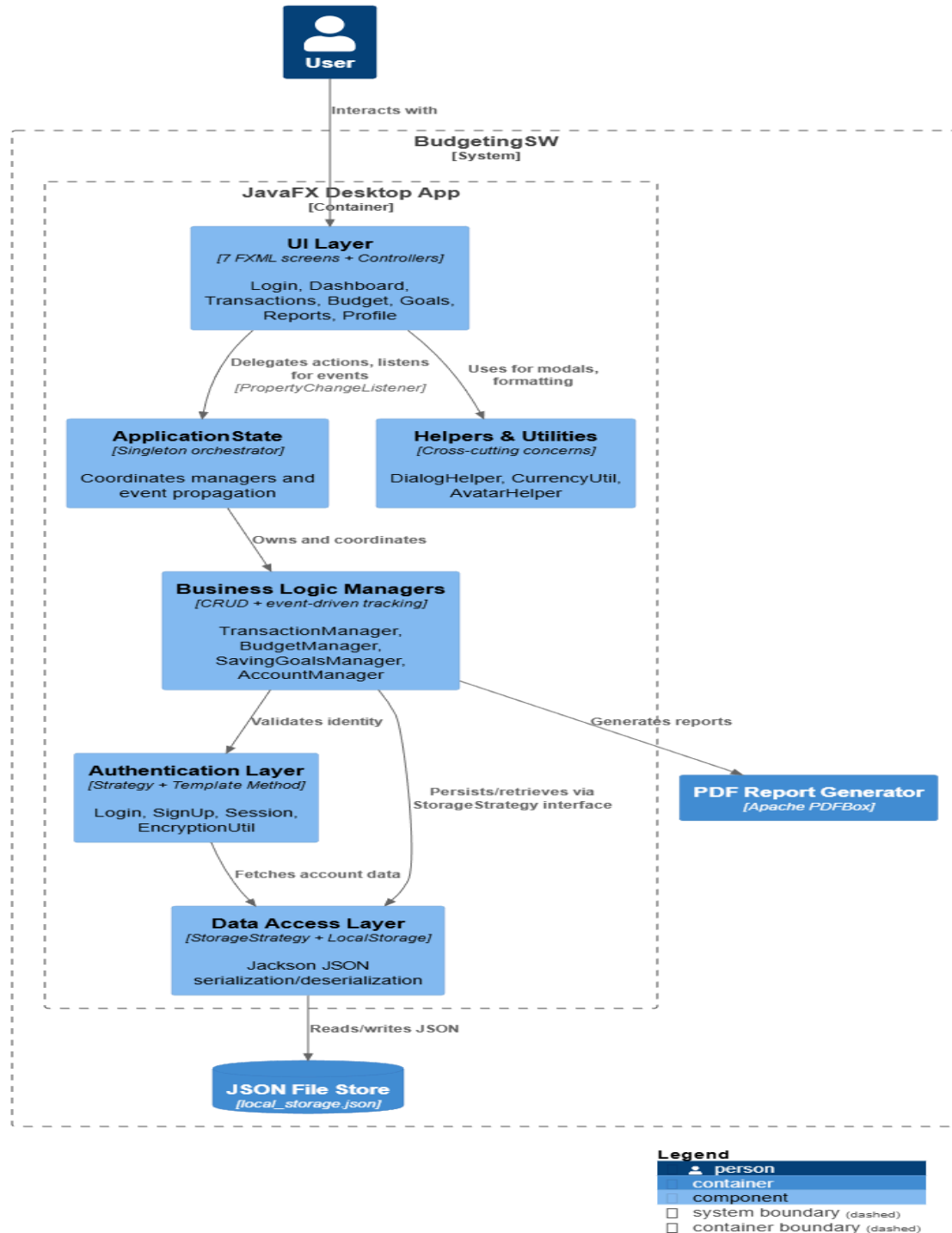




CS251: Ducks in Hell

Project: BudgetingSW

Software Design Specification



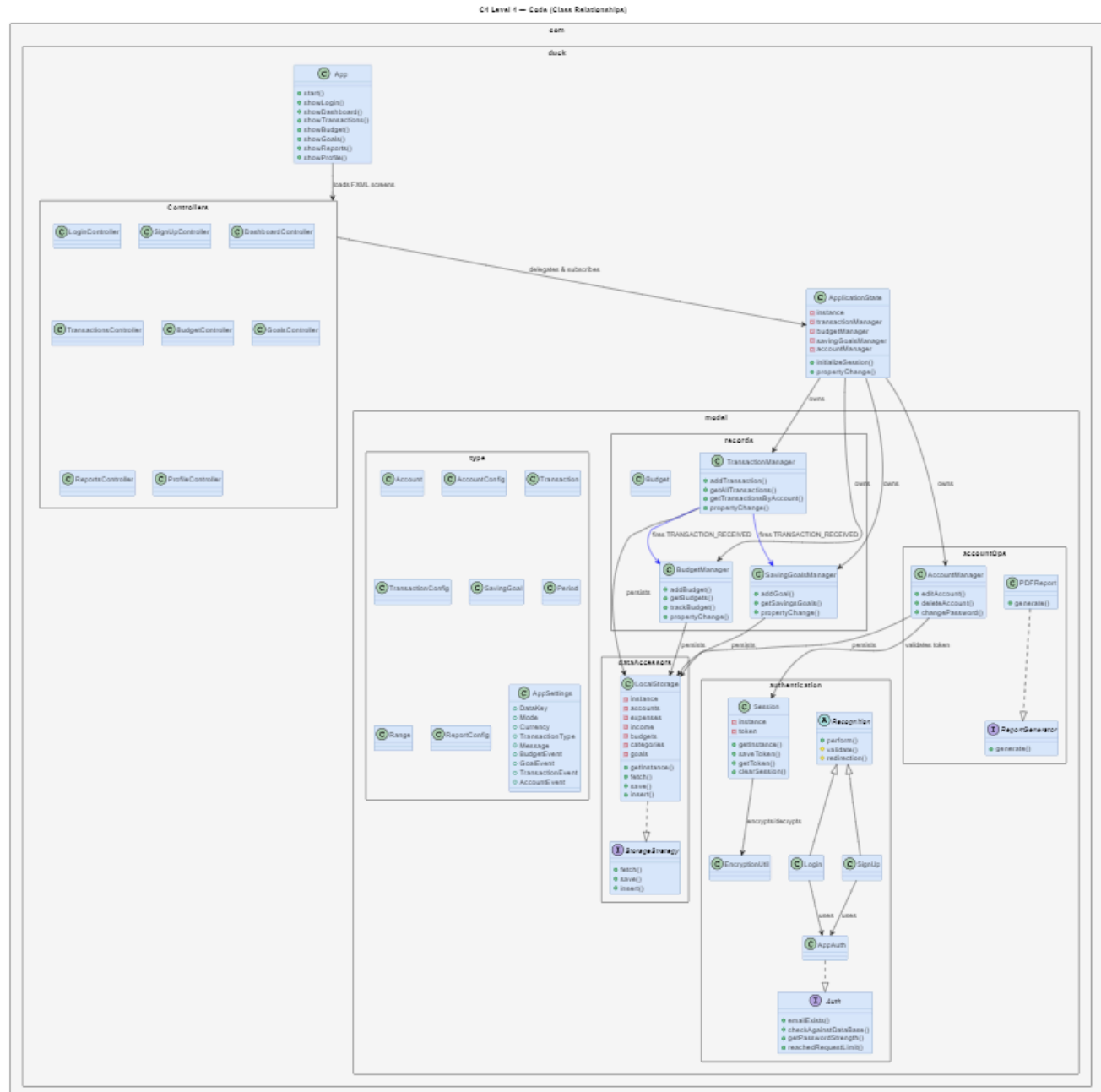


CS251: Ducks in Hell

Project: BudgetingSW

Software Design Specification

Software Design Specification





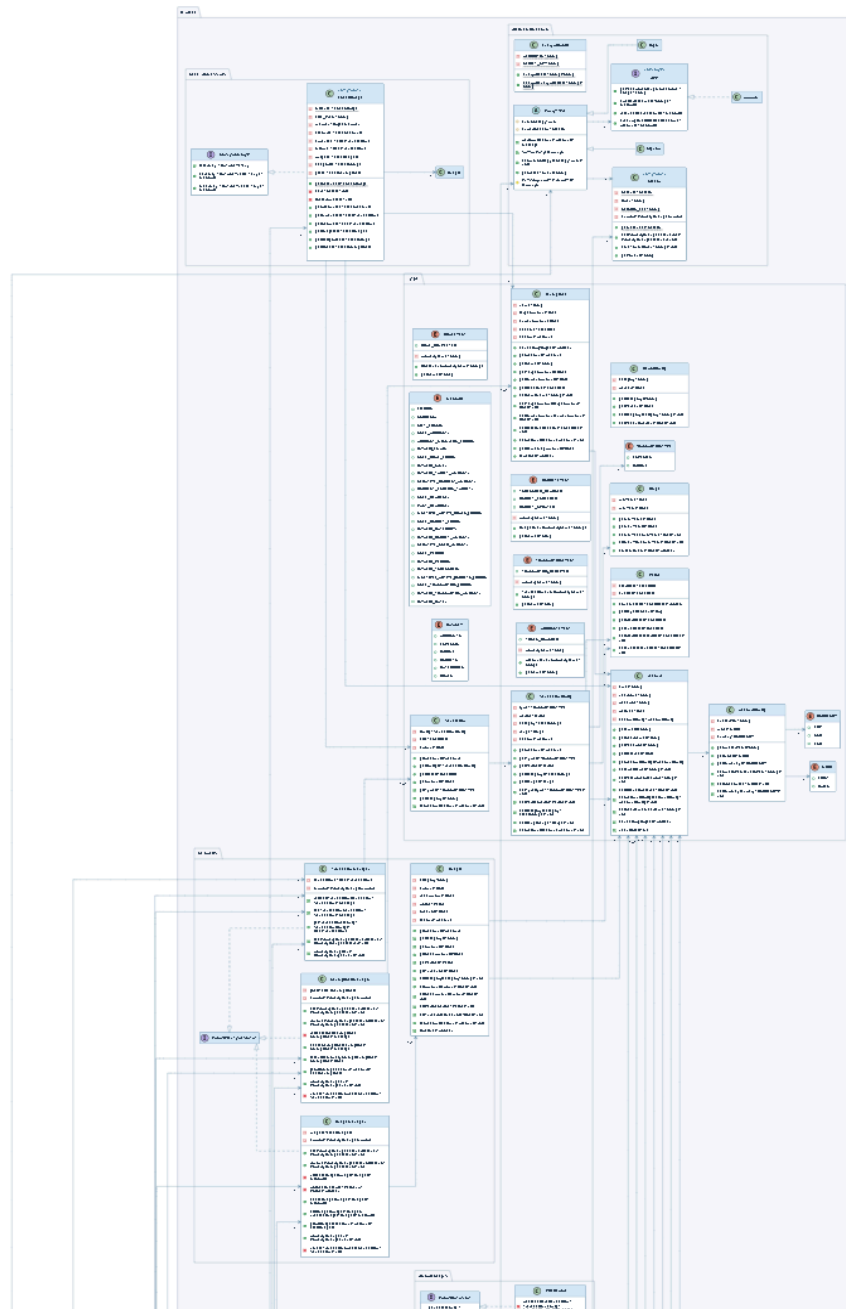
CS251: Ducks in Hell

Project: BudgetingSW

Software Design Specification

II. Class Diagram(s)

Better Quality: [Class Diagram](#)

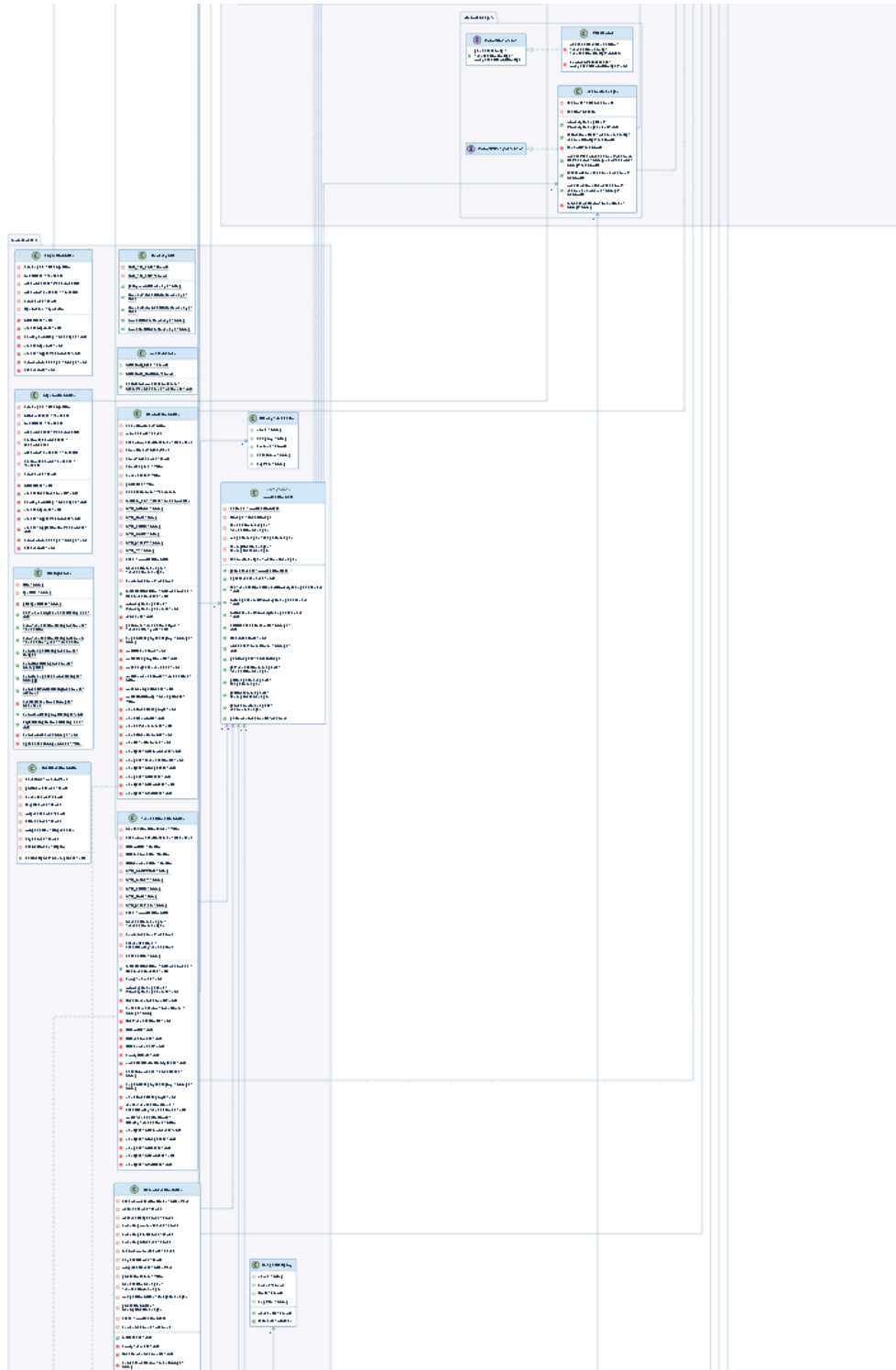




CS251: Ducks in Hell

Project: BudgetingSW

Software Design Specification

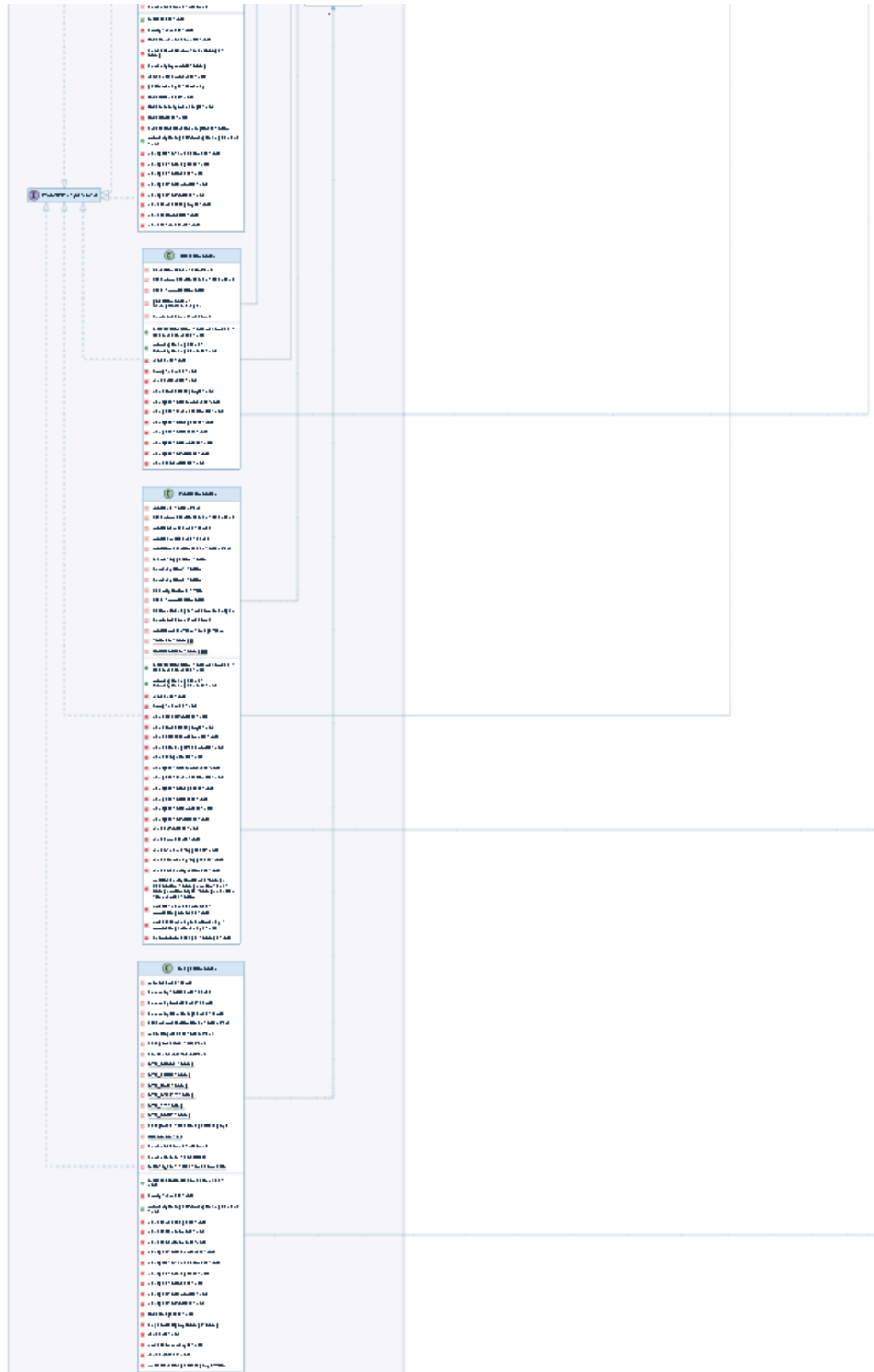




CS251: Ducks in Hell

Project: BudgetingSW

Software Design Specification





CS251: Ducks in Hell

Project: BudgetingSW

Software Design Specification

III. Class Descriptions

Class ID	Class Name	Description & Responsibility
1.	AccountManager	Manages profile updates (editing accounts). Implements PropertyChangeListener to update account balances.
2.	PDFReport	A concrete implementation of ReportGenerator that outputs data in PDF format.
3.	ReportGenerator	Defines the Strategy for generating financial reports based on transaction configurations.
4.	AppAuth	Manages user authentication logic, including verifying credentials and checking password strength using regex patterns. It utilises StorageStrategy to interface with account data.
5.	Auth (Interface)	Defines the Strategy for authentication checks and password strength
6.	EncryptionUtil	A utility class that provides static methods for securing data using the AES algorithm. It handles the transformation of plain text into Base64-encoded encrypted strings and vice versa using a predefined secret key.
7.	SignUp	Concrete implementation of Recognition , specifically handling new account creation validation.
8.	Login	Concrete implementation of Recognition , specifically handling user login validation.
9.	Recognition (Abstract)	Uses the Template Method pattern to outline the core flow (perform, redirect, validate) for user identification tasks (login/signup).
10.	Session	A singleton class that manages the current user's authentication state. It is responsible for saving and loading encrypted tokens to a local file (session.dat) and notifying the system of changes via PropertyChangeSupport .
11.	LocalStorage	A concrete implementation (Singleton) of StorageStrategy for saving data locally.
12.	StorageStrategy	Defines the contract for data persistence (fetch, save, insert) using the Strategy pattern.
13.	Budget	Represents a budget limit for a specific category and period, tracking the used amount against a threshold
14.	BudgetManager	Manages budget operations. Implements PropertyChangeListener to update budget usage when new transactions occur.
15.	TransactionManager	Acts as the Subject in the Observer pattern. Handles adding/retrieving transactions and notifies observers (controllers) upon changes.
16.	SavingGoalsController	Manages savings goal operations. Implements IObserver to track goal progress based on transactions.
17.	Account	The central user entity holds credentials, balance, contact info, and specific account configurations.
18.	AccountConfig	Holds user preferences such as display name, avatar, UI mode, and preferred currency.
19.	AppSettings	Serves as the central namespace and container for all application-wide constants and type definitions. It organises internal enums like Currency , Message , and DataKey to ensure consistent data handling across the entire system.
20.	AppSettings & AccountEvent	Defines observable events related to user accounts, specifically tracking when a security token has changed.



CS251: Ducks in Hell

Project: BudgetingSW

Software Design Specification

Class ID	Class Name	Description & Responsibility
21.	AppSettings & BudgetEvent	Manages the states for budget tracking, including notifications for received transactions, reached thresholds, and exceeded limits.
22.	AppSettings & Currency	Provides a standardised list of supported financial denominations, including EGP, USD, and EUR.
23.	AppSettings & DataKey	Acts as a set of unique identifiers for fetching data types such as Accounts , Expenses , Income , Budgets , and Goals .
24.	AppSettings & GoalEvent	Handles events related to financial goals, specifically triggering when a goal is marked as completed.
25.	AppSettings & Message	A comprehensive list of application status and error codes (e.g., SUCCESS , INVALID_NAME , NEGATIVE_USED_AMOUNT , PAST_DEADLINE) used for user feedback.
26.	AppSettings & Mode	Defines the visual theme settings for the application interface, supporting DARK and LIGHT modes.
27.	AppSettings & TransactionEvent	Manages events specific to individual entries, such as when a new transaction is received.
28.	AppSettings & TransactionType	Categorises all financial movements into two primary states: INCOME or EXPENSE.
29.	Range	Encapsulates a minimum and maximum numerical value (e.g., used for transaction filtering).
30.	ReportConfig	Represents a data point for generated reports, mapping a specific category to a percentage.
31.	SavingGoal	Tracks a user's financial goal, including target amounts, current progress, and deadlines.
32.	Transaction	Represents a single financial record, storing its type, date, amount, and category.
33.	TransactionConfig	Acts as a configuration/filter object containing criteria (type, period, category, range) for fetching specific transactions.
34.	Period	Defines a specific time frame with a start and end date.
35.	App	Acts as the main application bootstrap. It initialises the primary stage, manages the global Scene , and provides static utility methods for dynamically loading FXML resources and changing the UI root.
36.	ApplicationState	Acts as the central orchestrator for the application's runtime. It maintains single instances of core managers (Transactions, Budgets, Goals) and facilitates global data access and session token broadcasting.
37.	AvatarHelper	A specialised utility class for UI aesthetics. It handles the loading, scaling, and circular clipping of user profile images to ensure consistent avatar displays across the sidebar and headers.
38.	BudgetController	Controls the main budget management view. It dynamically populates category spending grids, updates summary labels (Total, Spent, Remaining), and listens for transaction events to refresh the UI.
39.	BudgetController& BudgetCategory	A data model representing a specific spending category. It calculates expenditure percentages and determines "over-budget" status for UI highlighting.



CS251: Ducks in Hell

Project: BudgetingSW

Software Design Specification

Class ID	Class Name	Description & Responsibility
40.	CurrencyUtil	Provides global financial utilities. It handles real-time currency conversion logic between USD, EUR, and EGP, and formats numerical floats into localised currency strings with appropriate symbols.
41.	DashboardController	The primary controller for the main landing view. It aggregates data from across the system to display the overall balance, spending trends, and a summary of active saving goals.
42.	DialogHelper	A utility class for standardising modal interactions. It manages the creation and styling of custom FXML-based dialogues, such as the transaction entry forms, and applies global CSS themes to them.
43.	GoalCardController	Manages the UI logic for individual saving goal widgets. It calculates the visual progress bar fill, determines the days remaining until a deadline, and updates the status labels for a specific goal.
44.	GoalsController	Manages the collective goals view. It acts as a container for GoalCardController instances, triggers the creation of new goals via dialogues, and coordinates the layout of all active saving targets.
45.	LoginController	Manages the authentication user interface. It captures user credentials from the view, triggers sign-in/sign-up logic via AppAuth , and handles UI feedback such as error messages and password visibility.
46.	ProfileController	Handles user settings and account customisation. It manages the UI for updating the user's name and email, toggling between Light and Dark modes, and switching the primary display currency (USD, EUR, EGP).
47.	ReportsController	Orchestrates the data visualisation layer. It processes financial data to populate the cash flow charts and donut charts, manages the month-selection logic, and provides a summarised breakdown of expenses and goal progress.
48.	SignUpController	Controls the account creation interface. It validates user input (matching passwords, email format), interfaces with the Recognition engine to register new users, and handles the transition to the login screen.
49.	TransactionController	Manages the transaction history view. It is responsible for filtering records (All, Income, Expense), dynamically generating the transaction list UI using SVG icons for categories, and handling the "Add Transaction" trigger.
50.	TransactionController&DisplayTransaction	A specialised Data Transfer Object (DTO) used to bind transaction data to the UI. It stores processed strings for dates and SVG paths for category icons to simplify list rendering.

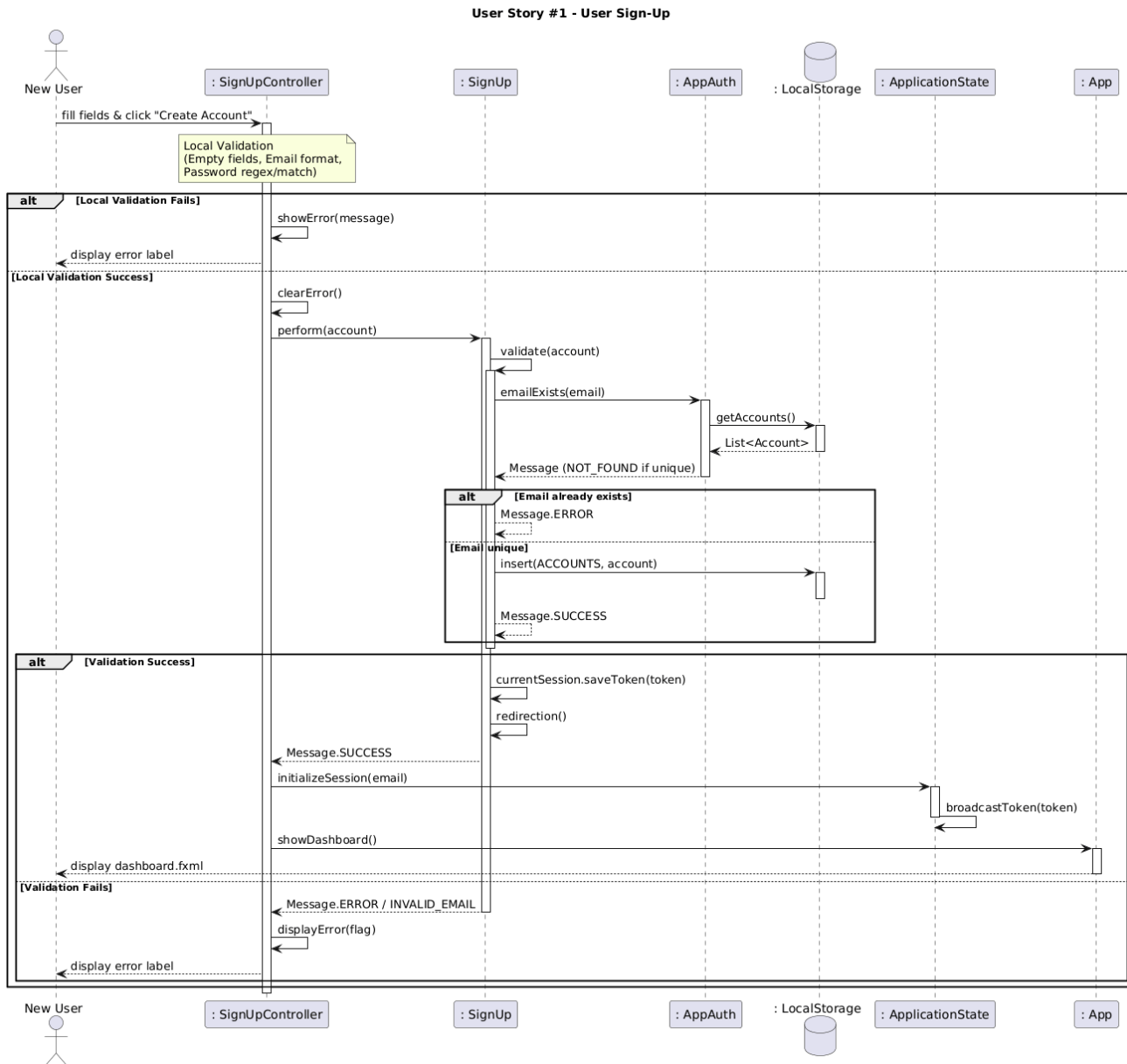


CS251: Ducks in Hell

Project: BudgetingSW

Software Design Specification

IV. Sequence diagrams

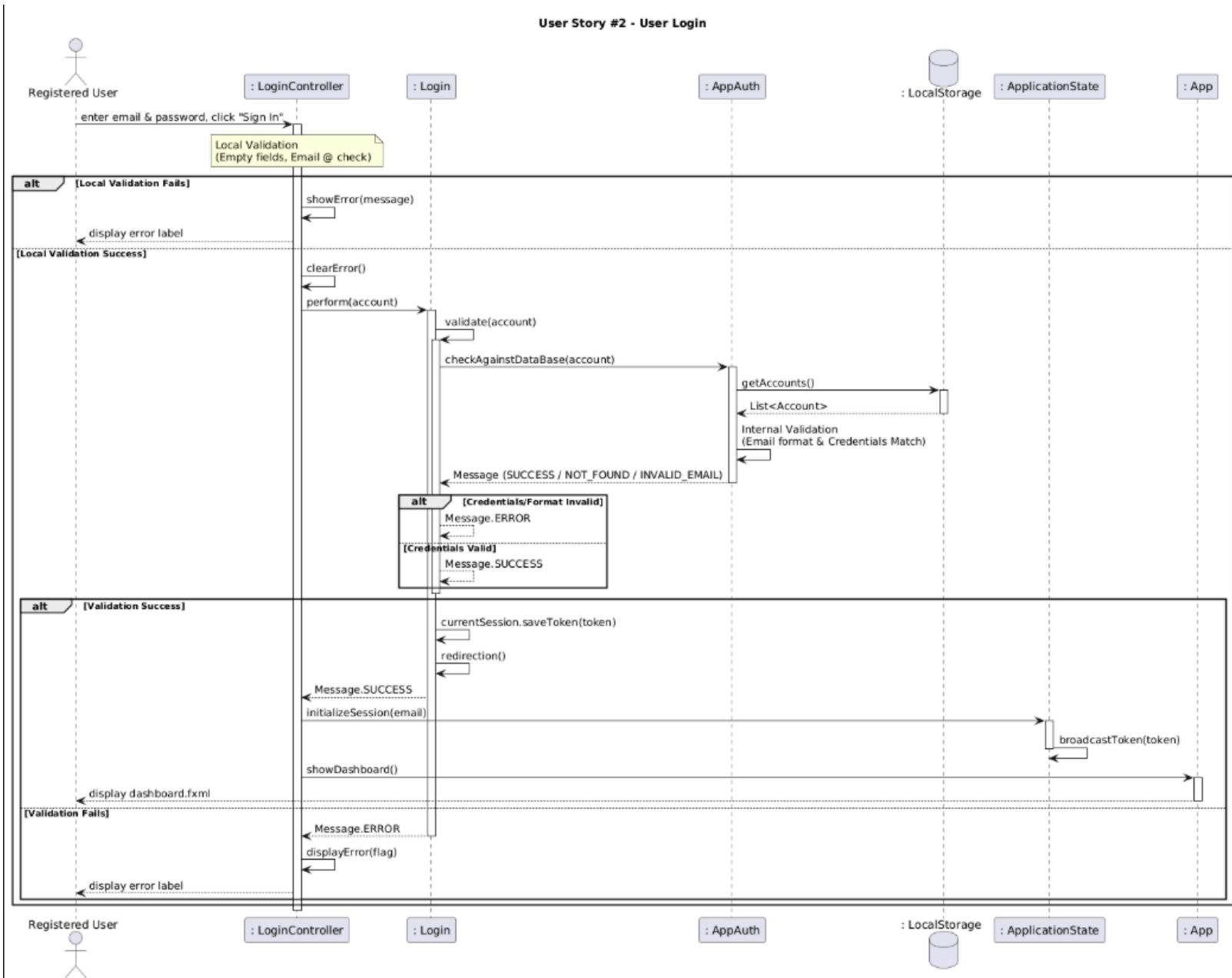




CS251: Ducks in Hell

Project: BudgetingSW

Software Design Specification

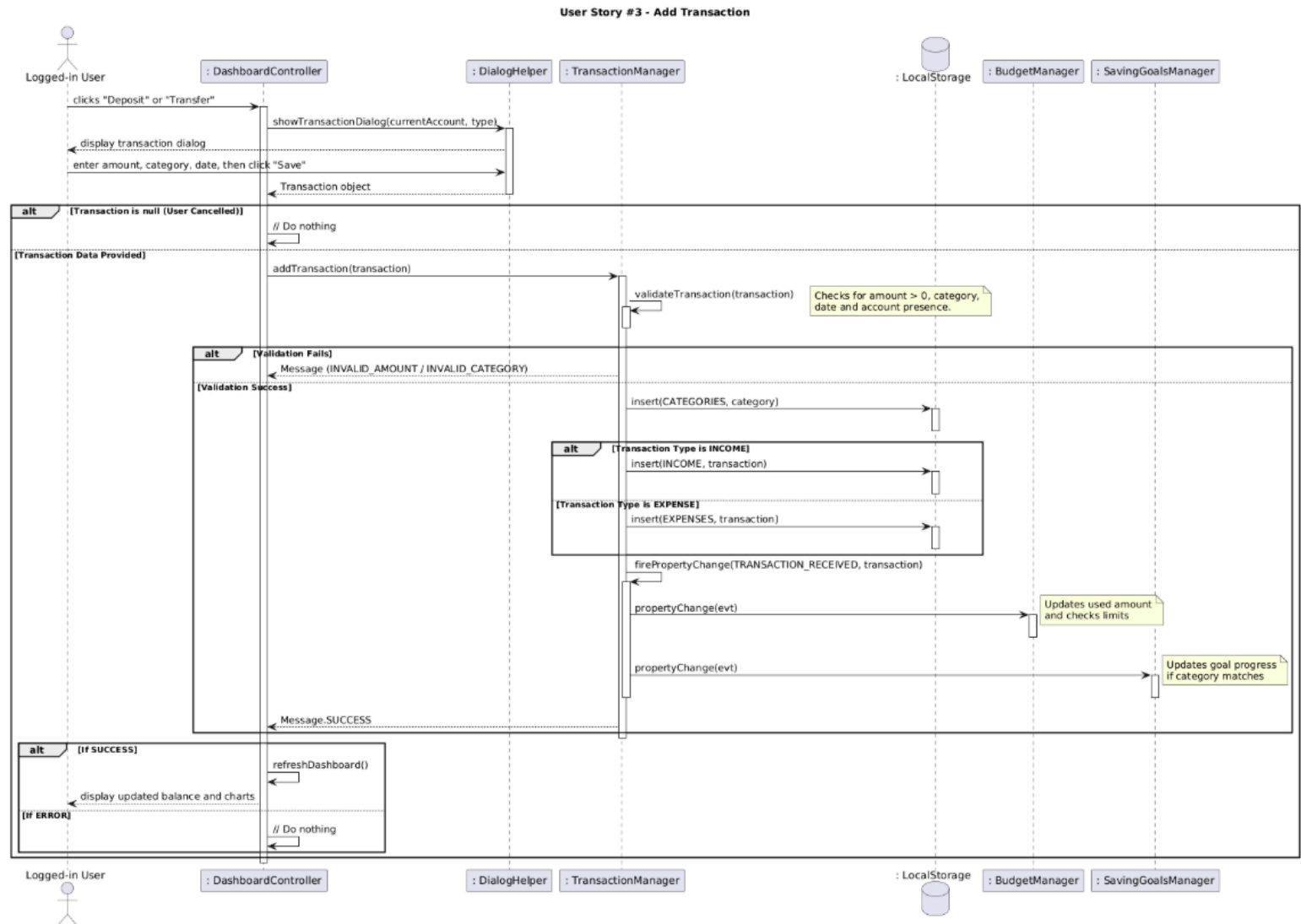




CS251: Ducks in Hell

Project: BudgetingSW

Software Design Specification

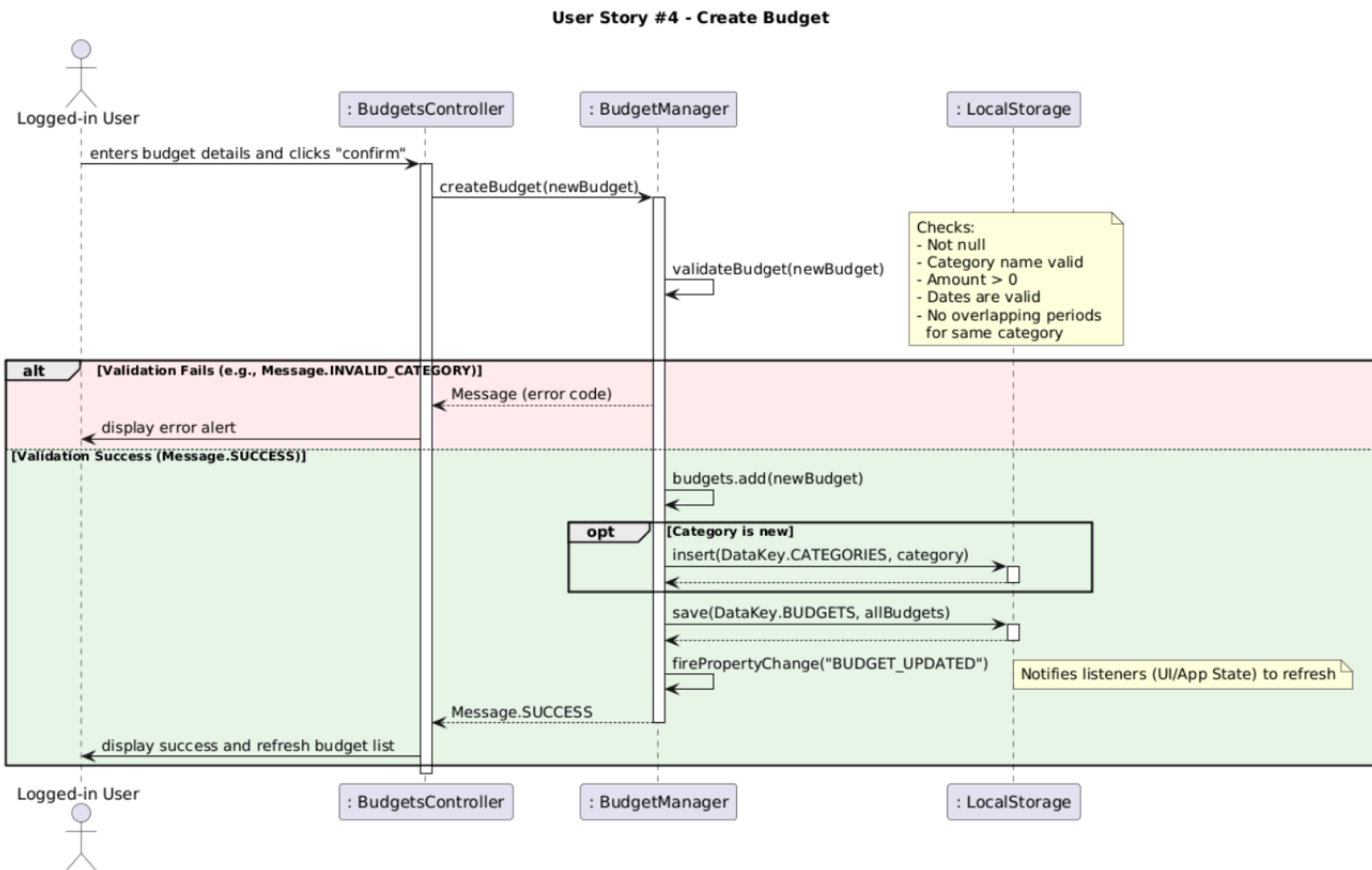




CS251: Ducks in Hell

Project: BudgetingSW

Software Design Specification

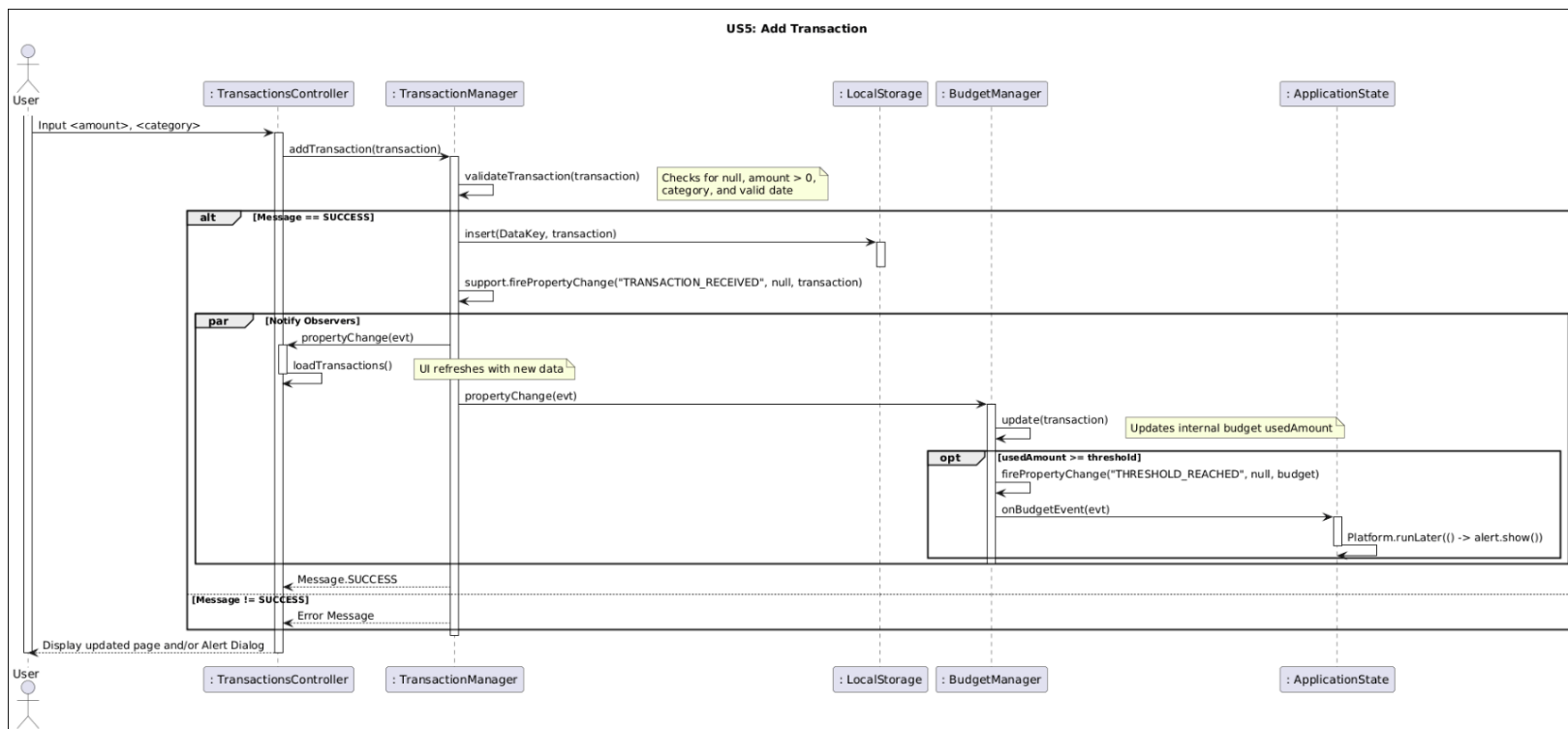




CS251: Ducks in Hell

Project: BudgetingSW

Software Design Specification



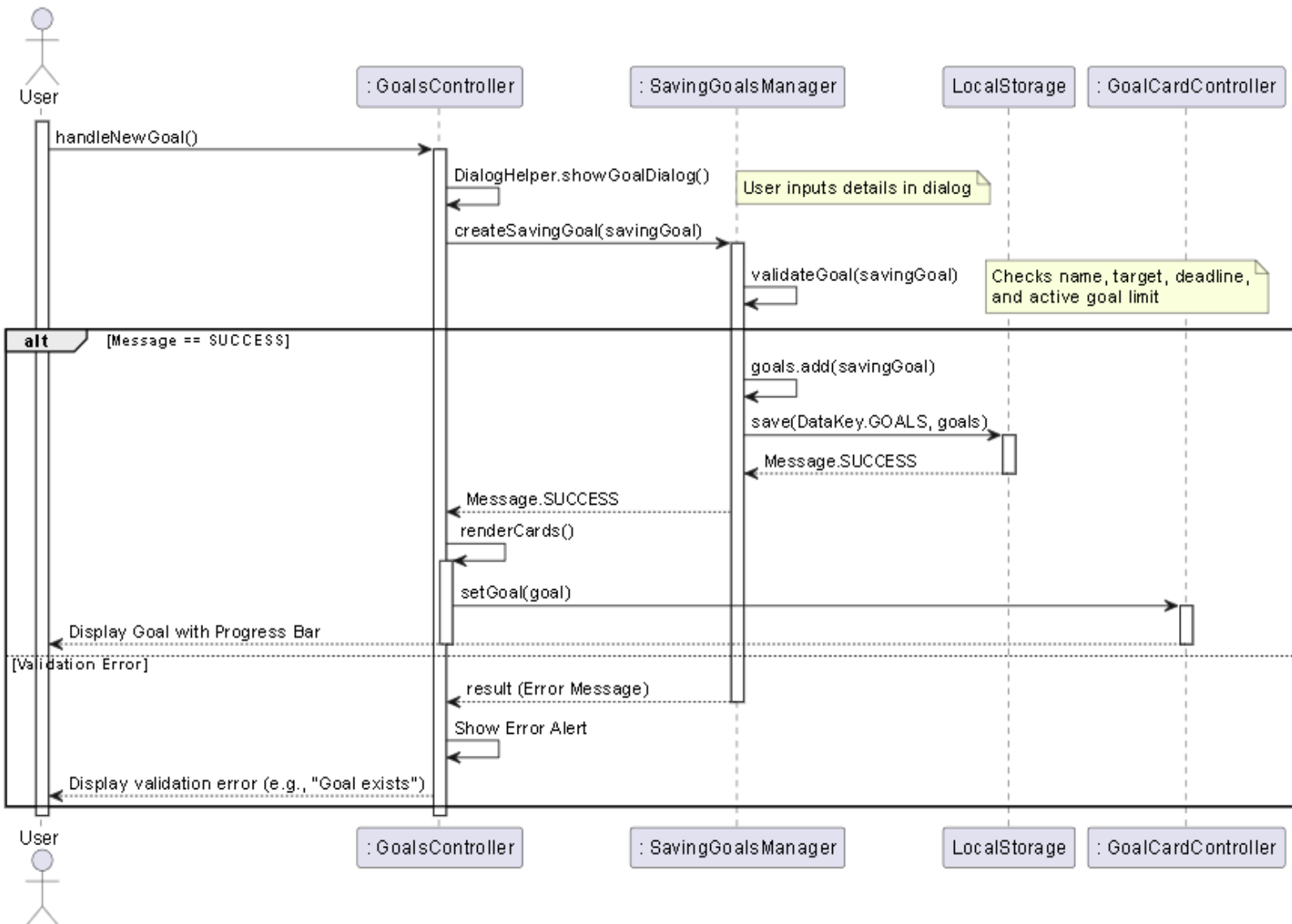


CS251: Ducks in Hell

Project: BudgetingSW

Software Design Specification

User Story #6 - Create Saving Goal



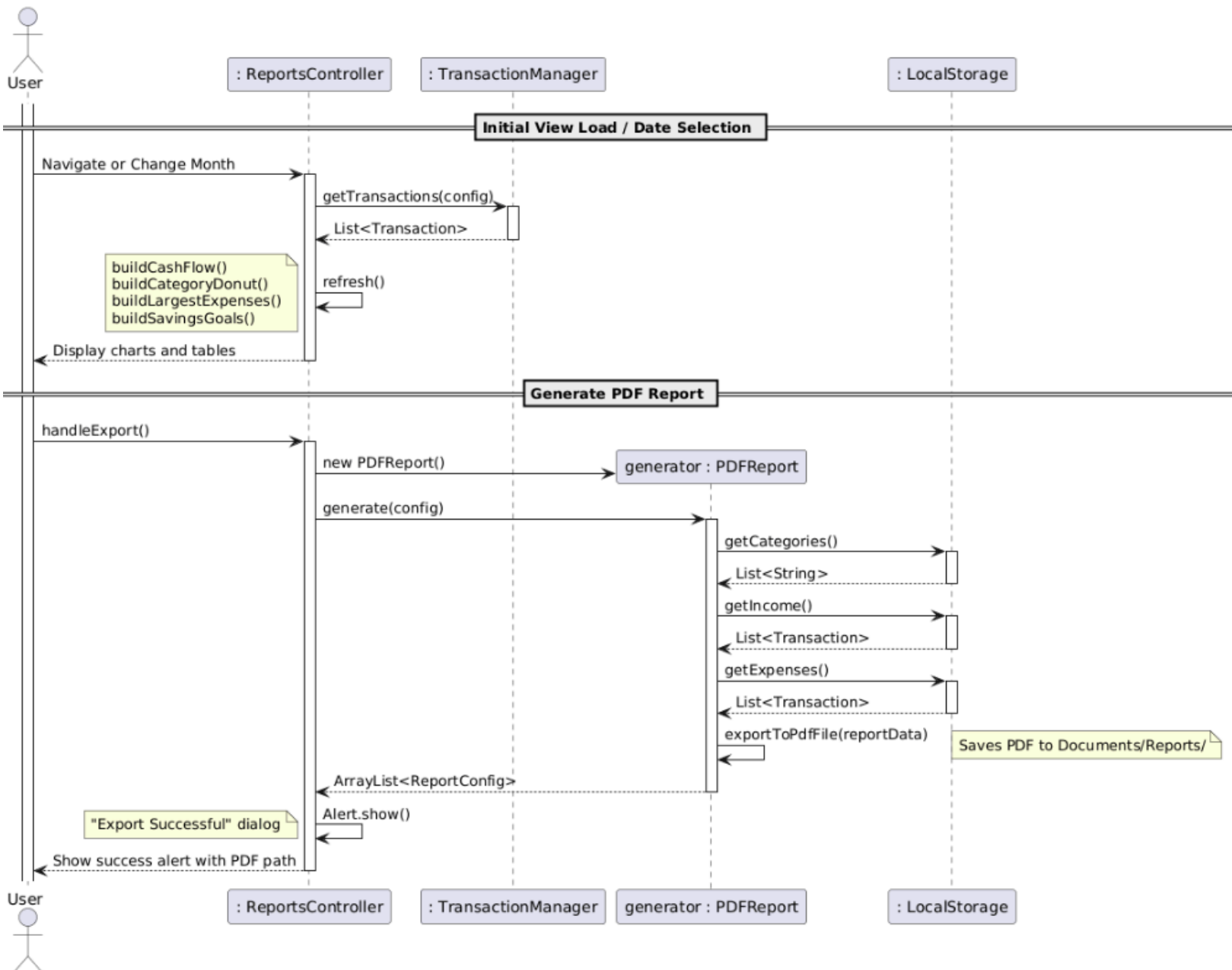


CS251: Ducks in Hell

Project: BudgetingSW

Software Design Specification

User Story #7 - Generate Financial Report





CS251: Ducks in Hell

Project: BudgetingSW

Software Design Specification

Class - Sequence Usage Table

Sequence Diagram	Classes Used	All Methods Used
1. User Sign-Up	- SignUpController - SignUp - AppAuth - LocalStorage - ApplicationState - App	- SignUpController::showError(message) - SignUpController::clearError() - SignUpController::displayError(flag) - App::showDashboard() - SignUp::perform(account) - SignUp::validate(account) - SignUp::redirection() - SignUp::currentSession.saveToken(token) - AppAuth::emailExists(email) - LocalStorage::getAccounts() - LocalStorage::insert(ACCOUNTS, account) - ApplicationState::initializeSession(email) - ApplicationState::broadcastToken(token)
2. login	- LoginController - Login - AppAuth - LocalStorage - ApplicationState - App	- LoginController::showError(message) - LoginController::clearError() - LoginController::displayError(flag) - Login::perform(account) - Login::validate(account) - Login::redirection() - Login::currentSession.saveToken(token) - AppAuth::checkAgainstDataBase(account) - LocalStorage::getAccounts() - ApplicationState::broadcastToken(token) - ApplicationState::initializeSession(email) - App::showDashboard()
3. Add Transaction	- DashboardController - DialogHelper - TransactionManager - LocalStorage - BudgetManager - SavingGoalsManager	- DialogHelper::showTransactionDialog(currentAccount, type) - DashboardController::refreshDashboard() - TransactionManager::addTransaction(transaction) - TransactionManager::validateTransaction(transaction) - TransactionManager::firePropertyChange(TRANSACTION_RECEIVED, transaction) - LocalStorage::insert(CATEGORIES, category) - LocalStorage::insert(INCOME, transaction) - LocalStorage::insert(EXPENSES, transaction) - BudgetManager::propertyChange(evt) - SavingGoalsManager::propertyChange(evt)
4. Create/Edit Budget	- BudgetsController - BudgetManager - LocalStorage	- BudgetManager::createBudget(newBudget) - BudgetManager::validateBudget(newBudget) - BudgetManager::budgets.add(newBudget) - BudgetManager::firePropertyChange("BUDGET_UPDATED") - LocalStorage::insert(DataKey.CATEGORIES, category) - LocalStorage::save(DataKey.BUDGETS, allBudgets)



CS251: Ducks in Hell

Project: BudgetingSW

Software Design Specification

Sequence Diagram	Classes Used	All Methods Used
5. Budget Over-Limit Alert	- TransactionsController - TransactionManager - LocalStorage - BudgetManager - ApplicationState	- TransactionsController::propertyChange(evt) - TransactionsController::loadTransactions() - TransactionManager::addTransaction(transaction) - TransactionManager::validateTransaction(transaction) - TransactionManager::support.firePropertyChange("TRANSACTION_RECEIVED", null, transaction) - LocalStorage::insert(DataKey, transaction) - BudgetManager::propertyChange(evt) - BudgetManager::update(transaction) - BudgetManager::firePropertyChange("THRESHOLD_REACHED", null, budget) - ApplicationState::onBudgetEvent(evt) - ApplicationState::Platform.runLater(() -> alert.show)
6. Set and Track Financial Goals	- GoalsController - SavingGoalsManager - LocalStorage - GoalCardController	- GoalsController::handleNewGoal() - GoalsController::DialogHelper.showGoalDialog() - GoalsController::renderCards() - GoalsController::Show Error Alert - SavingGoalsManager::createSavingGoal(savingGoal) - SavingGoalsManager::validateGoal(savingGoal) - SavingGoalsManager::goals.add(savingGoal) - LocalStorage::save(DataKey.GOALS, goals) - GoalCardController::setGoal(goal)
7. View Reports and Analytics	- ReportsController - TransactionManager - PDFReport - LocalStorage	- ReportsController::getTransactions(config) - ReportsController::refresh() - ReportsController::handleExport() - ReportsController::Alert.show() - TransactionManager::getTransactions(config) - PDFReport::generate(config) - PDFReport::exportToPdfFile(reportData) - LocalStorage::getCategories() - LocalStorage::getIncome() - LocalStorage::getExpenses()

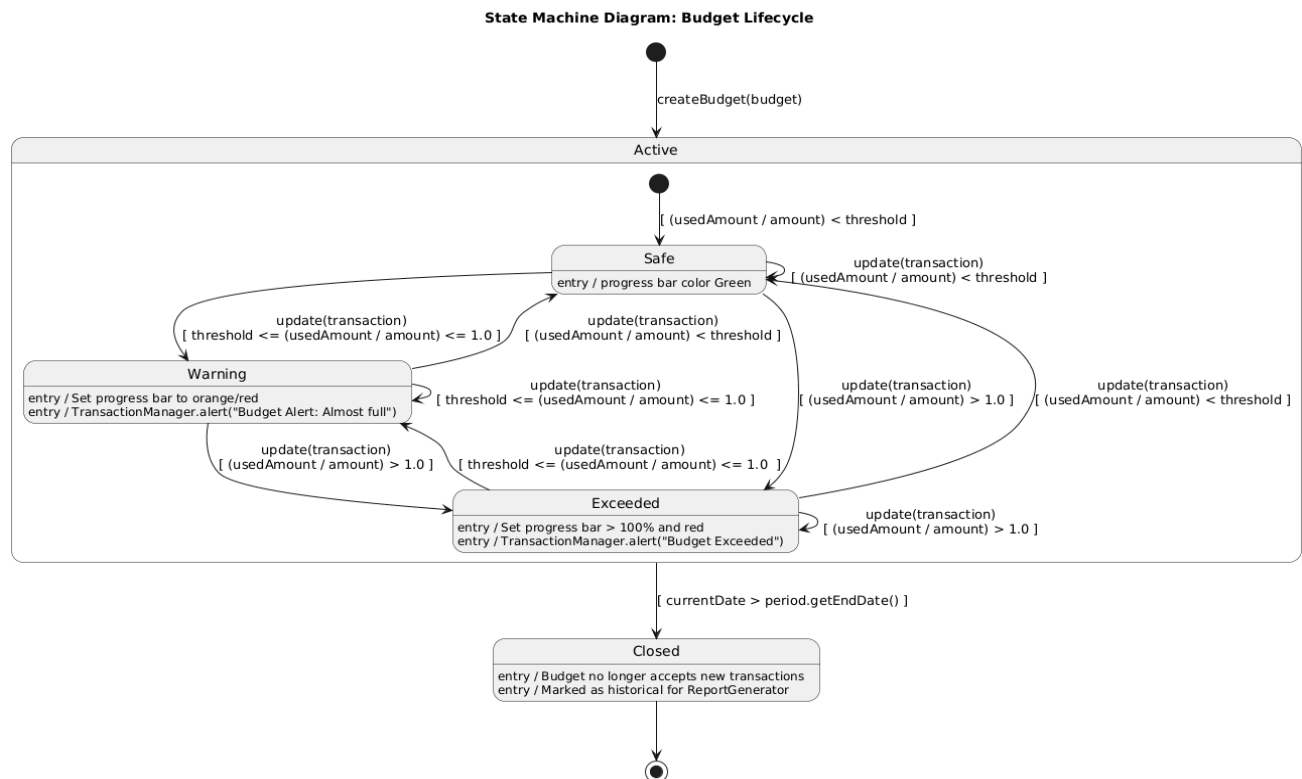


CS251: Ducks in Hell

Project: BudgetingSW

Software Design Specification

V. State Diagram





CS251: Ducks in Hell

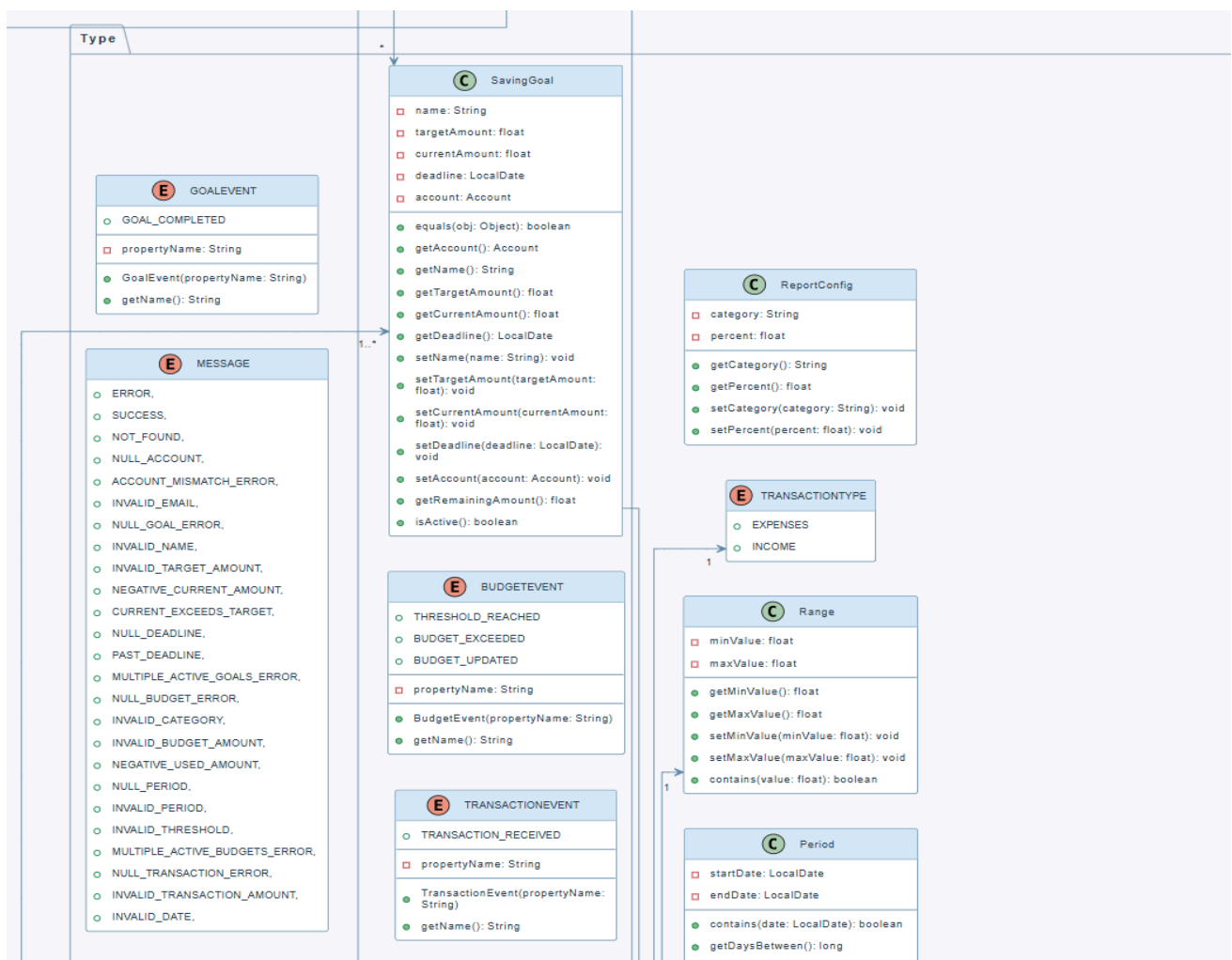
Project: BudgetingSW

Software Design Specification

VI. SOLID Principles

A. Single Responsibility Principle

The Architecture depends on the single responsibility principle by dividing into different classes; each class has one responsibility to be changed, like the Account class, which exists only to represent a client account, where the Range exists to define {min, max}, and the MESSAGE to represent successful or errors (with its different types).



B. Open/Close Principle

CU – FCAI – CS251 Introduction to Software Engineering – 2026 - Software Design Specifications

Prepared by Mostafa Saad and Mohammad El-Ramly V1.0

Edited by Mohamed Samir, Updated to V2.0 by Mohammad El-Ramly 10/4/2020 and V3.0 25/5/2021, 20/4/2025

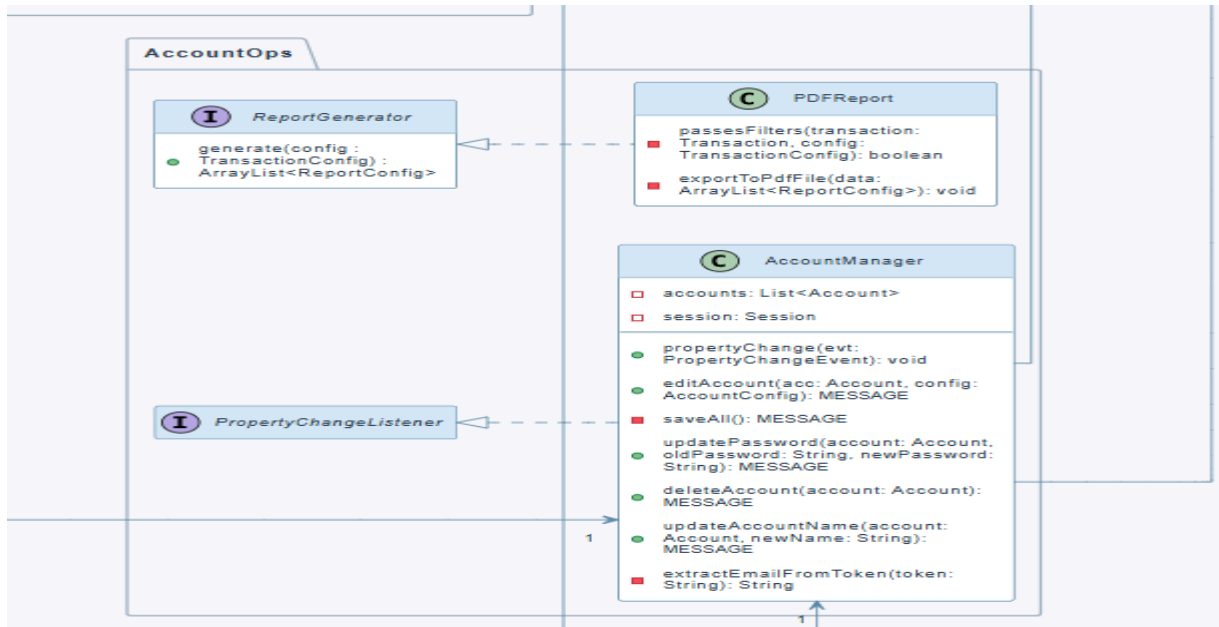


CS251: Ducks in Hell

Project: BudgetingSW

Software Design Specification

The Architecture used the Open/Close Principle by using the Strategy Pattern in different locations in the design, such as the Report Generator, which is a template interface for generating reports with specific formats, and StorageStrategy, which provides a unified access using an interface.





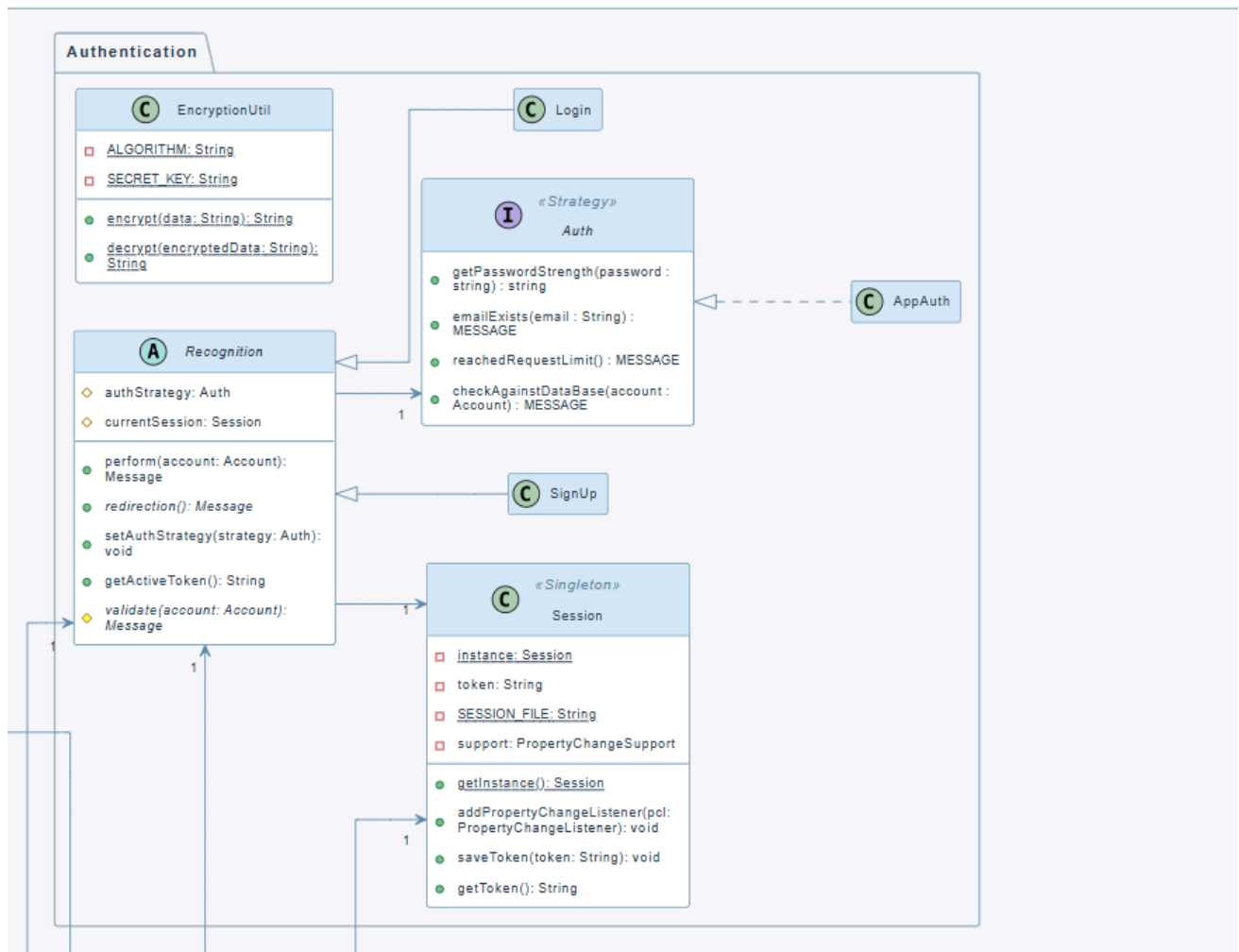
CS251: Ducks in Hell

Project: BudgetingSW

Software Design Specification

C. Dependency Inversion

Data Accessors and Authentication depend on interfaces, not concrete classes, such as Recognition, which depends on Auth, where Auth is an interface.





CS251: Ducks in Hell

Project: BudgetingSW

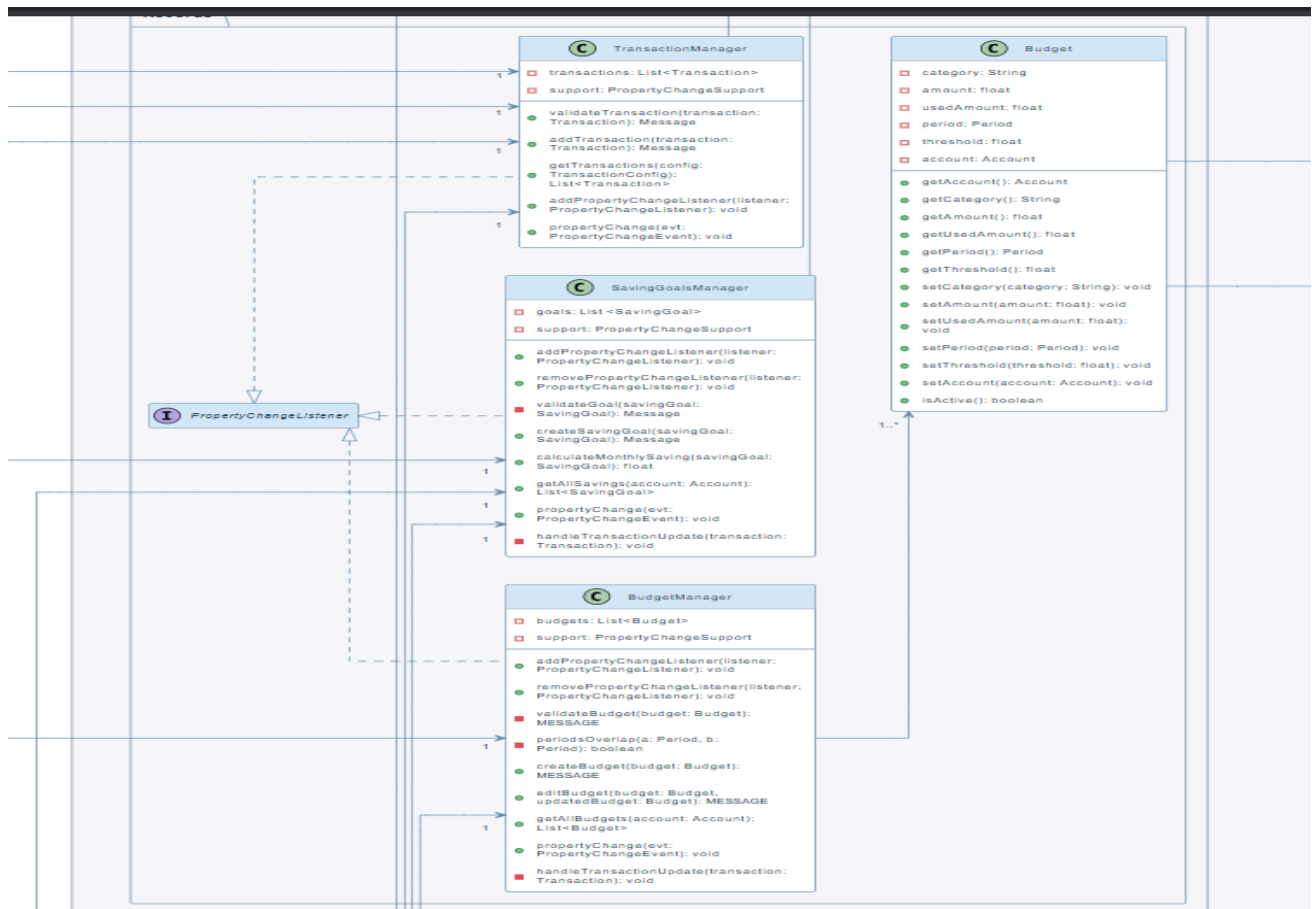
Software Design Specification

VII. Design Patterns

A. Observer Pattern

In the **Records** package. The **TransactionManager** acts as the subject that broadcasts the notification to the observers, where they are **SavingGoalsController**, **BudgetController**, and other controllers.

Benefits: Adding new transaction delegates its update states automatically without any complex set of processes





CS251: Ducks in Hell

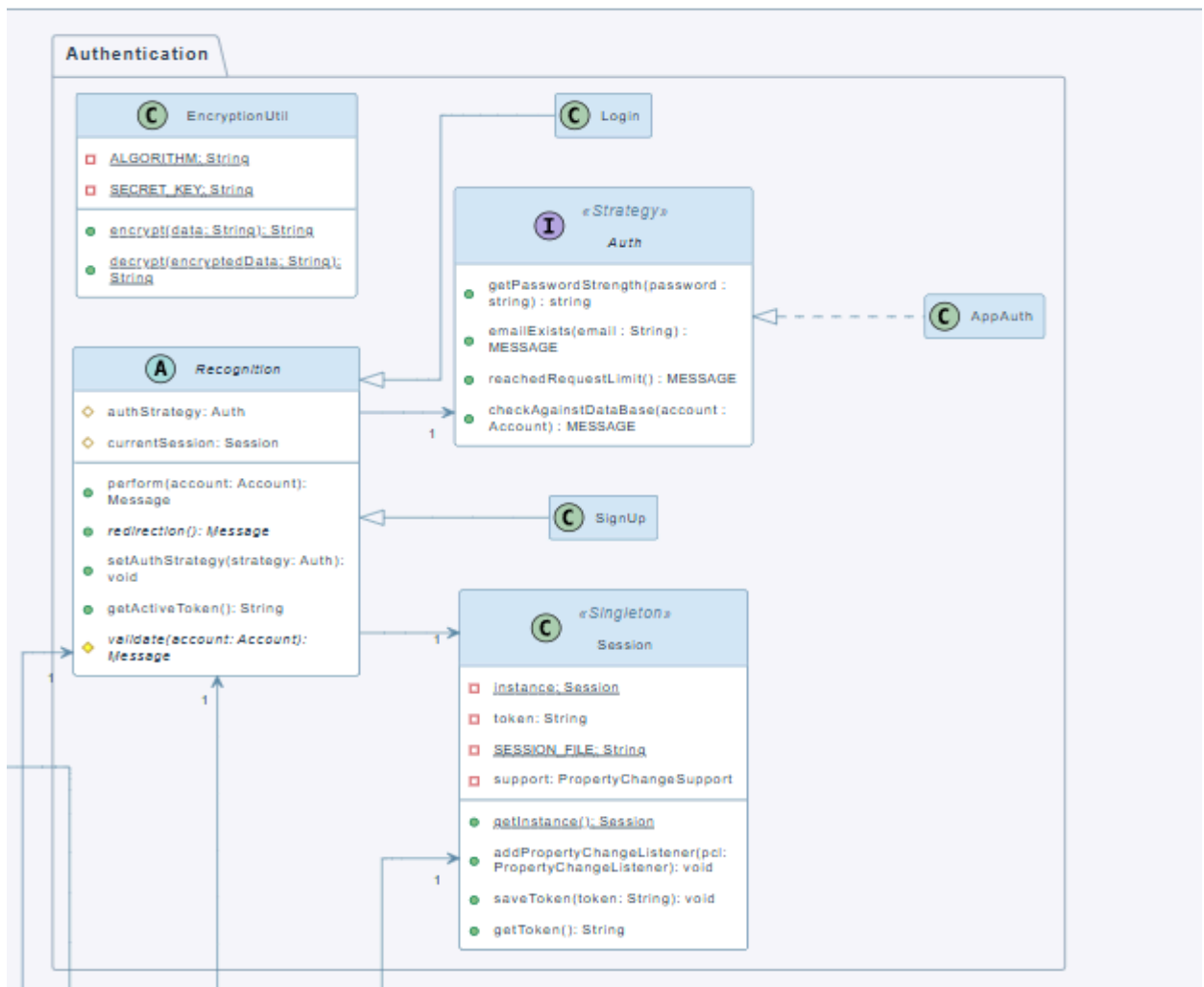
Project: BudgetingSW

Software Design Specification

B. Template Method Pattern

In the **Authentication** Package with the abstract class **Recognition** class.

Benefits: The Template Method defines the skeleton of the main function of the class Page, Recognition, so that each child of it complies with the rules that have been set, minimising developer mistakes





CS251: Ducks in Hell

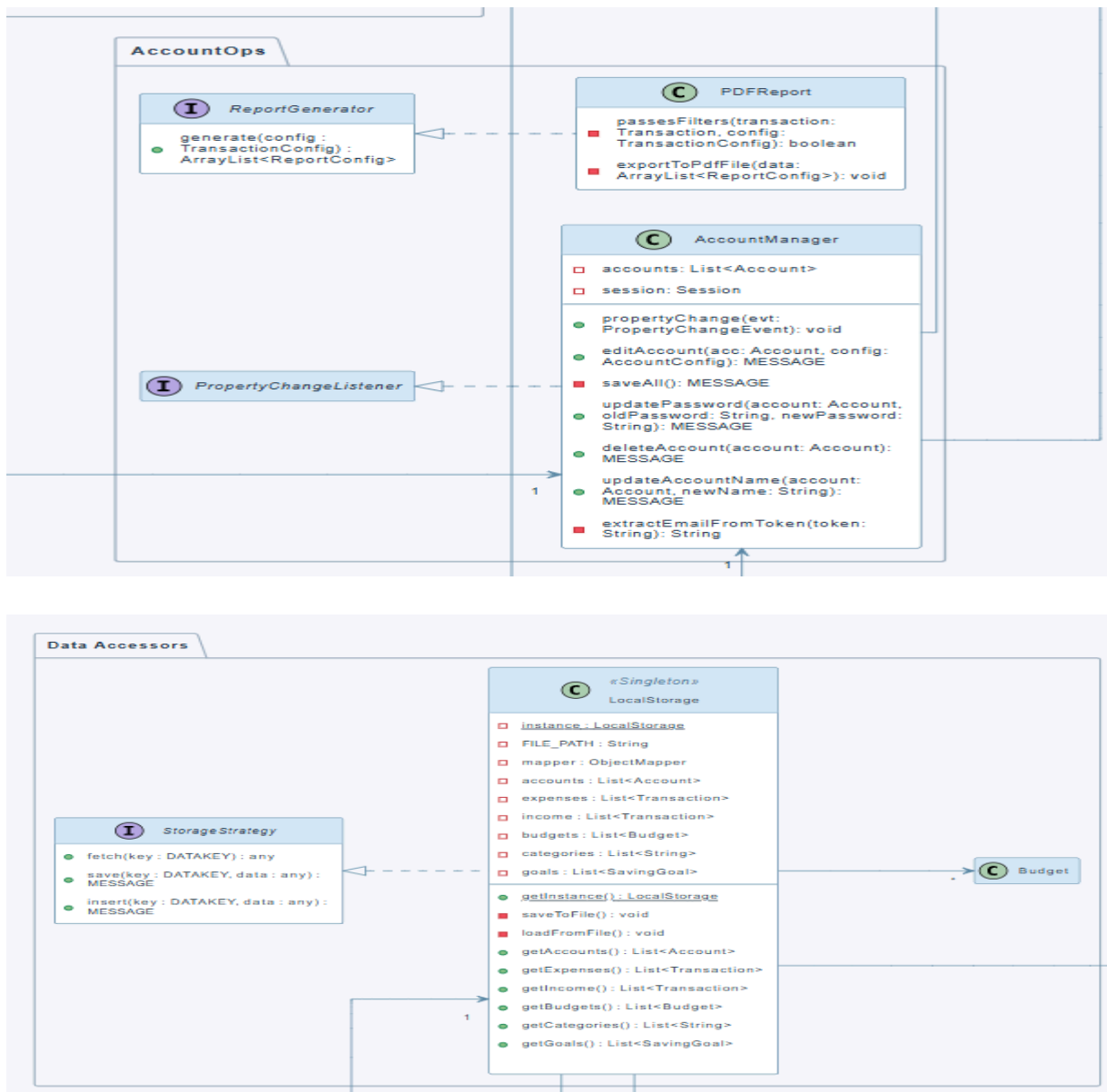
Project: BudgetingSW

Software Design Specification

C. Strategy Pattern

In the **Data Accessors** Package with the **StorageStrategy** interface, and in the **Authentication** Package with the abstract class **Auth** interface.

Benefits: Preserving the Open/Closed Principle by providing an extendable object by implementing the interface without any change in the client code





CS251: Ducks in Hell

Project: BudgetingSW

Software Design Specification

Tools

- PlantUML
- VS code
- JavaFX
- Maven

Ownership Report

Item	Owners
Architectural Diagram	Ali Wael, Youssef Ibrahim, Mohamed Ayman, Omar Abdelmonem
Class Diagram	Ali Wael, Youssef Ibrahim, Mohamed Ayman, Omar Abdelmonem
Sequence Diagram (1 to 4)	Mohamed Ayman, Omar Abdelmonem
Sequence Diagram (5 to 7)	Ali Wael, Youssef Ibrahim
State Diagram	Ali Wael, Youssef Ibrahim
Solid Principles	Ali Wael
Design Patterns	Ali Wael, Mohamed Ayman